Comprehensive RAG Evaluation and Model Deployment

Introduction

Imagine the HR policy assistant at Brightpath Solutions gives a confident but incomplete answer about contractor leave. Did the retriever fetch the wrong documents, or did the model write a weak answer from good ones? Without the right metrics, you are guessing.

This reading completes the four **RAGAS** (Retrieval-Augmented Generation Assessment) metrics and explains why the RAGAS demo called a cloud API while the capstone runs its model locally. You will learn which half of a pipeline to fix when a score drops, and how to handle API keys safely.

An Analogy: The Grocery Run and the Kitchen

Retrieval is the grocery run; generation is the cooking.

→ Context precision asks how much of your shopping bag you actually used. Buy five, use two: low precision.

→ Context recall asks whether you bought everything the recipe needed. Forget the eggs, and the dish fails however well you cook.

→ Faithfulness asks whether the cook used only what was in the bag.

→ Answer relevancy asks whether the cook made the dish that was ordered.

Context Precision and Context Recall

Scoring Generation Faithfulness and Relevance with RAGAS showed faithfulness and answer relevancy. The remaining two metrics judge the retriever.

Context precision measures how many retrieved chunks were useful. RAGAS also rewards ranking relevant chunks near the top. Irrelevant chunks waste prompt space and distract the model.

Context recall measures whether the retrieved chunks covered everything needed. Suppose a user asks about leave for contractors in the London office, including notice periods. If your retriever finds the leave chunk but misses the notice-period chunk, recall is low, even though everything retrieved was accurate.

Metric	Stage	Low score means	Look first at
Faithfulness	Generation	Unsupported claims added	Prompt and grounding rules
Answer relevancy	Generation	Answer drifted off-topic	Prompt design
Context precision	Retrieval	Too many irrelevant chunks	Re-ranking, smaller top-k
Context recall	Retrieval	Needed information missed	Hybrid or parent-document retrieval

```
7
8
9
10
11
```

```
```

RAGAS uses a language model to judge relevance and weighs rank order, so real scores differ.

Two Ways to Deploy a Language Model

The RAGAS demo called Groq, a cloud inference provider, with an API key. The capstone uses Ollama, which runs the model on your own computer.

Factor	Groq (cloud API)	Ollama (local)
Where the model runs	Provider servers	Your computer
Internet needed	Yes	No, after download
API key	Yes	No
Cost	Pay per token (illustrative; check current pricing)	Free after download
Speed	Fast, network dependent	Depends on your hardware

Cloud APIs suit quick prototypes without a **GPU** (graphics processing unit). Local serving suits offline work and data that must stay on your machine.

Handling API Keys Safely

```
1
2
3
4
5
6
7
8
```

Add .env to your .gitignore file so the key never reaches a shared repository. Providers also enforce rate limits and show usage costs in a dashboard.

Weighted Sums and Reciprocal Rank Fusion

The hybrid search demo merged scores with a weighted sum: 60 percent dense, 40 percent keyword. RAG Best Practices and Comparisons described **Reciprocal Rank Fusion (RRF)**, which scores each document by 1/(k + rank) in each list, with k often set to 60. A weighted sum needs both scores normalized first. RRF skips that step, which helps when BM25 and cosine similarity produce very different ranges. Both are valid.

Common Pitfalls

→ Tuning the prompt when recall is the problem. A model cannot use a chunk that was never retrieved.

→ Committing a .env file. A leaked key must be revoked and replaced.

→ Expecting context recall without references. RAGAS compares retrieved context with a reference answer, so you need known good answers.

Conclusion

RAGAS scores tell you where a fault lives: generation metrics point to your prompt, retrieval metrics to your retriever. In Capstone: Building a Document-Based Knowledge Assistant, you will combine retrieval, chunking, and grounded generation in one assistant, running locally through Ollama.

Key Takeaways

→ Context precision checks for junk in the pile; context recall checks for missing pieces.

→ Cloud APIs trade per-token cost for convenience; local serving trades speed for control.

→ Keep API keys in environment variables, never in code.